PYTHON • AWS • WEB

FLYTON

DEVELOPMENT METHODOLOGY

A consistent method for building web applications
for entrepreneurs at the Innovation Center

Simple code. Clear infrastructure. Repeatable process.

FLYTON

Based on the Flyton repository and the workflow details provided

Why Flyton?

A shared methodology helps entrepreneurs and developers move faster—without reinventing the infrastructure for every project.

Maintainability

Readable code, focused functions, and reusable Flyton core utilities.

Consistency

Shared conventions for folders, naming, sessions, APIs, and database access.

Simplicity

Python 3, runtime-generated HTML, and minimal reliance on external packages.

Goal: reduce operational complexity and keep the focus on the product and the entrepreneur.

Technology Stack

Frontend

HTML • CSS • JavaScript | Simple UI; JavaScript when interactivity is needed

Flyton Core

Python 3 • CGI • APIs | Routing, HTML generation, sessions, and shared utilities

Web Server

Apache2 • Ubuntu Linux | CGI runs Python code on the server

Data

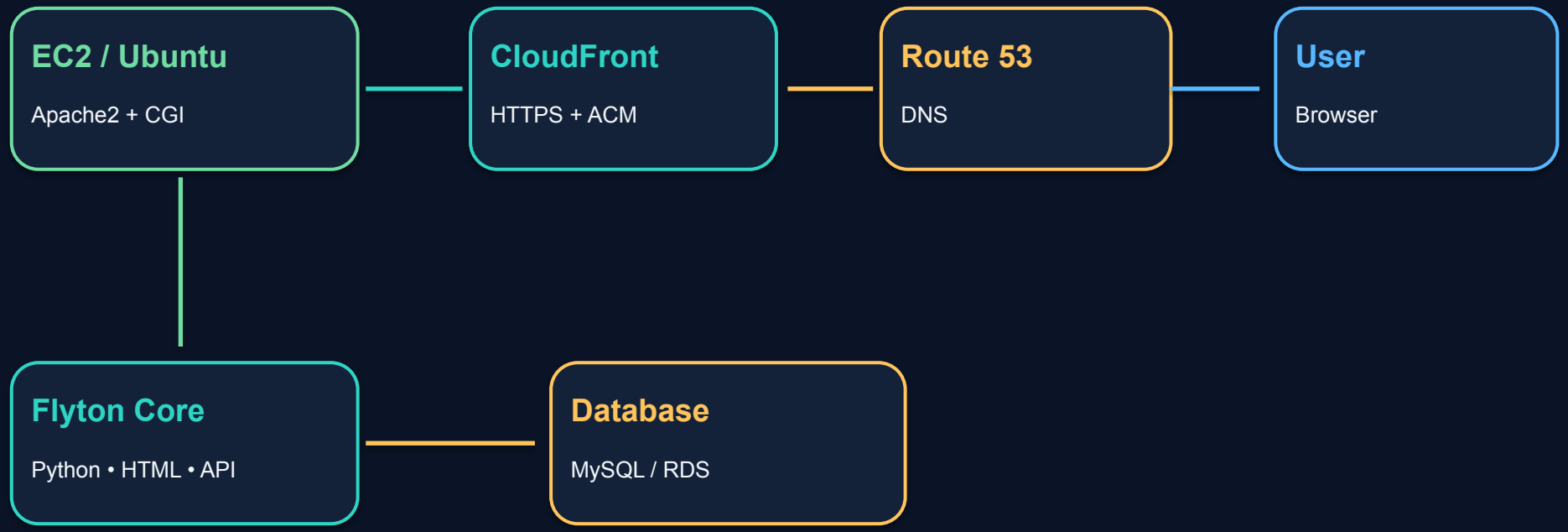
MySQL or Amazon RDS | Consistent access through Flyton SQL utilities

Cloud

AWS EC2 • CloudFront • Route 53 | Hosting, HTTPS, and DNS

Database choice varies by project: MySQL on Linux or Amazon RDS.

Production Architecture



HTML response returns through the same path

Note: Separately ensure that origin access rules restrict traffic according to security policy.

AWS Cloud9 - IDE

Real-Time Collaborative Coding

Share your development environment with team members in just a few clicks. Pair program, review code in real-time, and chat without leaving the IDE.

Consistent Pre-configured Envs

Eliminate "works on my machine" issues. Standardize runtimes, SDKs, and tooling across the entire engineering team with identical cloud environments.

Direct AWS Service Integration

Pre-authenticated terminal access with sudo privileges to the underlying EC2 instance. Native support for AWS CLI, Lambda debugging, and serverless tools.

Browser-Based Flexibility

Code from anywhere using any modern web browser. Auto-hibernate capability saves cost when inactive while state and files persist seamlessly on EC2.

Cloud9 streamlines developer onboarding and enables seamless collaboration while maintaining tight security integration with AWS resources.

Development Environment: AWS Cloud9

PROD

Production is separate; per the stated rules, administrative access to production is only through the development environment.

DEV

Development work takes place in Cloud9; code and the development workflow are managed from this environment.

Separation of Development and Production

What we do not assume

No automated CI/CD is assumed; the stated process is Git followed by manual deployment.

What happens in development?

Code editing • Testing • Git • Deployment preparation

Operational note: AWS Cloud9 availability varies by account and Region; this deck describes the development environment selected for this methodology.

Standard Project Structure

Client-side code and static assets	**client/**
Interfaces and application areas	**admin/ • weba/**
Application pages	**pages/**
Shared JavaScript and CSS files	**pages/lib/**
Files and images	**pages/files/ • im/**
Server-side logic	**server/**
CGI entry point	**cgi-bin/**
Server-side API handlers	**apis/api/**
Shared utilities and processes	**apis/tools/**

Principle

Clear separation of UI, server-side logic, APIs, and static assets.

A consistent structure speeds up onboarding and simplifies maintenance across projects.

How a Page Is Generated in Flyton

01 — Browser
Request to /cgi-bin/p with rpage and session

02 — CGI entry
Apache runs the Python script

03 — Flyton core
Load the application and prepare data

04 — Page parts
Header → dynamic body → footer

05 — Response
HTML is generated with Python f-strings and sent to the browser

A page is split into at least a header, body, and footer; the body is selected by rpage.

Static pages can be served separately without running the Flyton core.

Coding Standards That Create Consistency

Separation of Concerns

Keep static JavaScript and CSS in separate files; use JavaScript when interactivity is needed.

HTML with f-strings

Keep conditionals outside strings; build readable, well-structured HTML.

Simple Python

No heavy framework; avoid new package dependencies unless necessary.

Few Classes

Small, focused functions fit the runtime page-generation model.

Readable Code

Prefer the simplest solution; avoid tricks and unnecessary complexity.

Naming

Use lowercase file and function names; the entry-point function matches the file name.

Principle: developers should be able to understand another project's code quickly.

Database Access Through a Shared Layer

Flyton centralizes data operations in shared utilities to improve consistency and reduce scattered SQL in application code.

Writes

insert_to_sql() and dedicated utilities for inserting and updating data.

Iterative reads

sql_order() — Iterate through results using yield.

Targeted reads

find_in_sql() — Find records using conditions.

Project-specific configuration

MySQL on Linux or Amazon RDS, depending on the project.

Archiving

Records are generally marked inactive rather than routinely deleted.

Table conventions

id, name, is_active, timestamps, and data JSON field per project rules.

Sessions, Authorization, and APIs

Authorization

Check roles and permissions on sensitive pages; never rely only on hiding UI elements.

API

The client sends a method and data object; Python returns a structured response or an error.

Sessions

The ses table manages active sessions; page data is exposed through a data object.

Important Rules

- Per Flyton rules, do not use cookies or localStorage to manage sessions.
- The session is stored in the ses table; the session ID is a token, not an integer.
- Use Flyton's API and navigation mechanisms instead of inventing a new layer.

Security and Operations: Principles and Checks

Production access

According to the stated policy: production administration is performed only from the development environment.

IAM

Access AWS resources following least privilege permissions only.

TLS / HTTPS

CloudFront + ACM certificate; Route 53 manages DNS.

Origin protection

Verify that EC2 is not directly exposed to the public beyond what is required.

Backups

Automated backups were confirmed as in use; verify backup frequency and restoration procedures.

CloudWatch

Monitoring and logs, according to the project's configuration.

Important: using a service does not prove that all controls are configured; verify settings in each account and project.

Project Delivery Workflow

1 | Define the need

Entrepreneur's goal, users, and desired outcome

2 | Create the foundation

Flyton structure, screens, data, and permissions

3 | Develop

Python, HTML, APIs, and database following the conventions

4 | Test

Test scenarios, permissions, errors, and data

5 | Git

Version control and release preparation

6 | Manual deployment

Deploy to production from the development environment and verify afterward

Automated CI/CD is not part of the currently described workflow and is not presented as an existing process.

FLYTON METHODOLOGY

One method.

More focus on the product.

Shared infrastructure and clear conventions help the team and entrepreneurs build, learn, and move forward—without unnecessary complexity.

Code

Simple, readable, and modular

Cloud

Core services and clear boundaries

Process

Git + Manual deployment

Sources: github.com/yaweli/flyton • README.md • flyton-prompt-rules.md